



CHƯƠNG 2: NÂNG CẤP VÀ KHÔI PHỤC ẢNH

Tiết 6: Thực hành Lab 2.2 - Tích chập
(Convolution) & Khôi phục ảnh

JetBrains Mono

Môn học: Thị giác máy tính (INF03111)

Giảng viên: Trần Thành Thắng

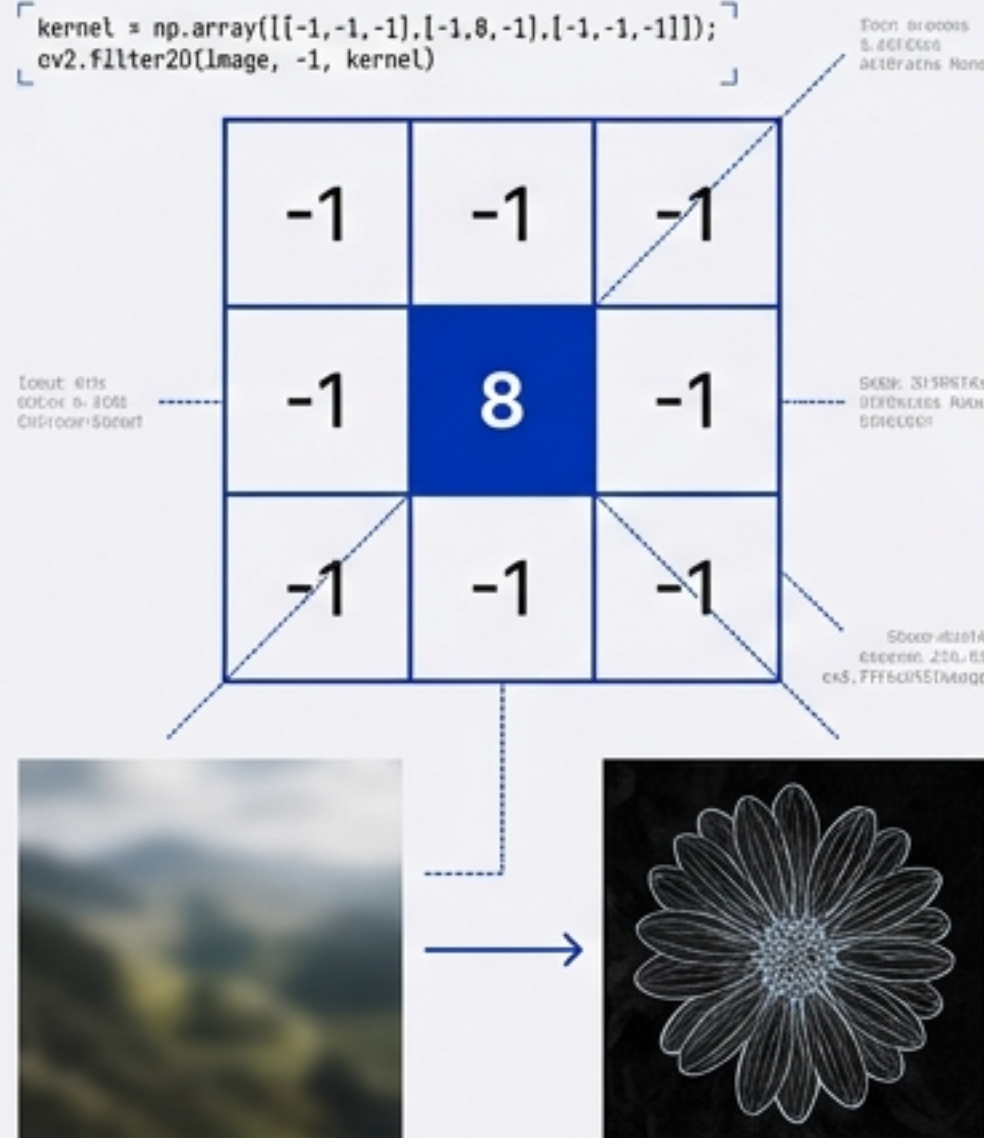
Lớp: IT23M

Nhiệm vụ Lab 2.2: Làm chủ Ma trận

Task 1: Custom Kernel

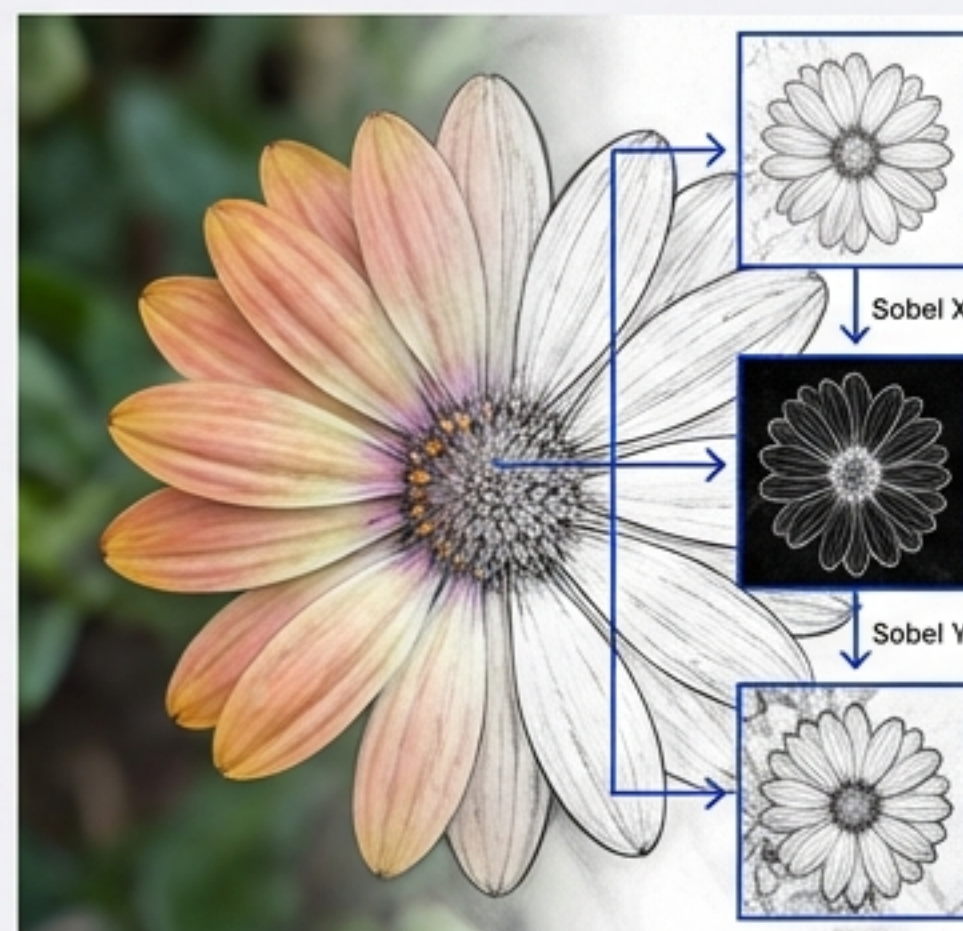
Tự định nghĩa ma trận và áp dụng `cv2.filter2D`.

```
kernel = np.array([[-1,-1,-1],[-1,8,-1],[-1,-1,-1]]);  
cv2.filter2D(image, -1, kernel)
```



Task 2: Edge Detection Art

Biến ảnh chụp thành tranh vẽ chì bằng bộ lọc Sobel.



```
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0);  
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1);  
sketch = cv2.bitwise_not(sobel_magnitude)
```

Task 3: Denoise Challenge

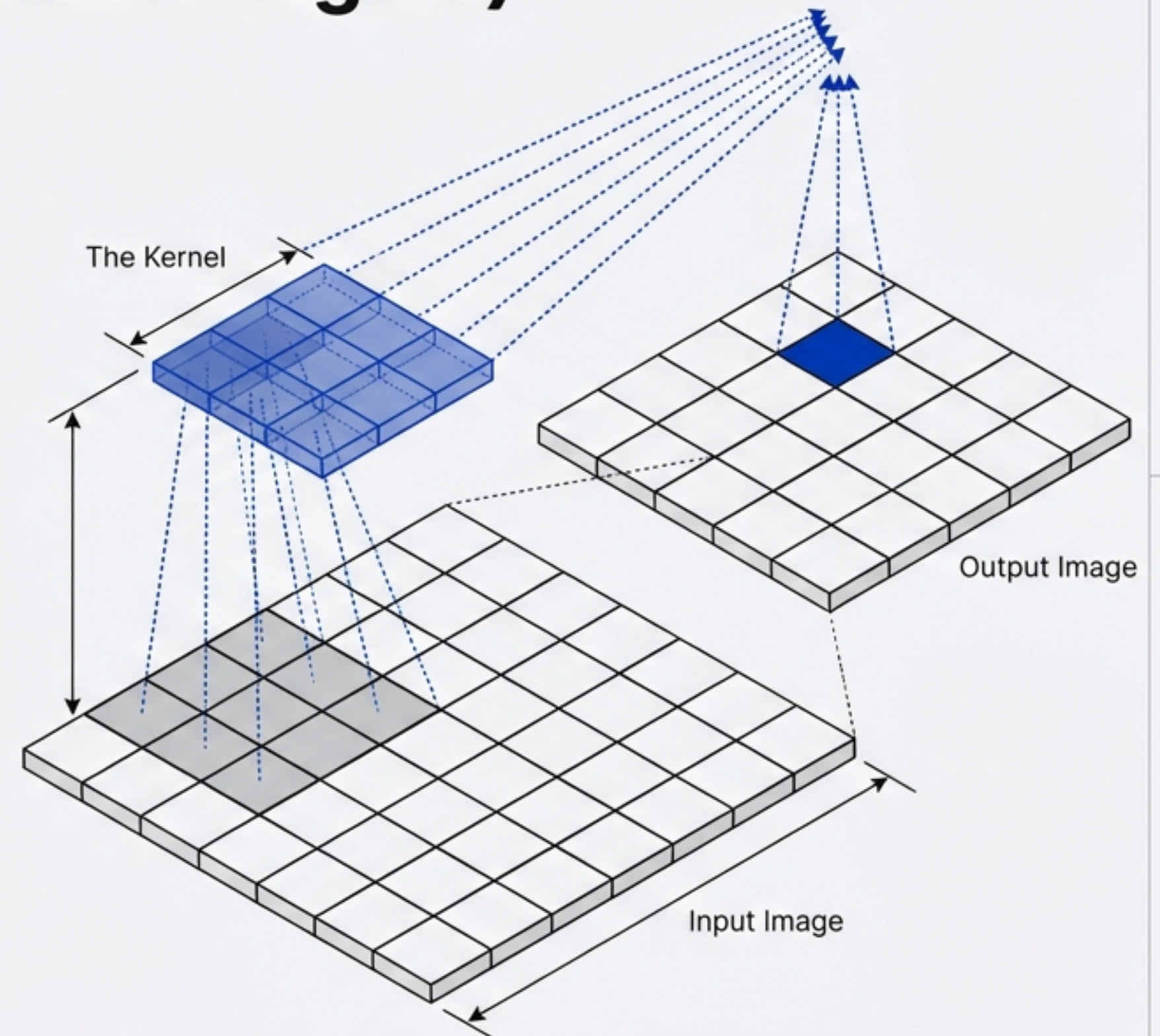
Phối hợp đa tầng lọc để "giải cứu" một bức ảnh thảm họa.



```
denoised_1 = cv2.medianBlur(noisy_img, 5);  
denoised_2 = cv2.fastNlMeansDenoisingColored(denoised_1);  
final = cv2.bilateralFilter(denoised_2, 9, 75, 75)
```


Cơ chế Tích chập (Convolution Engine)

- **Neighborhood Operations:** Xử lý dựa trên 'tình làng nghĩa xóm', không phải từng điểm độc lập.
- **Sliding Window:** Cửa sổ trượt quét qua toàn bộ ảnh.
- **Formula:** Giá trị pixel mới = Tổng trọng số (Pixel lân cận $\times$ Kernel).



Bài tập 1: Tự định nghĩa Kernel

```
kernel = np.array([[0, -1, 0],  
                  [-1, 5, -1],  
                  [0, -1, 0]])  
  
output = cv2.filter2D(src, -1, kernel)
```

0	-1	0
-1	5	-1
0	-1	0

Input



Output (Sharpened)



Bạn kiểm soát ma trận, bạn kiểm soát hiệu ứng thị giác.

Bài tập 2: Từ Toán học đến Nghệ thuật



Original



Grayscale



Sobel Gradient



Inverted (Sketch)

Quy trình: Sobel (Bắt biên) -> Magnitude (Tổng hợp) -> Invert (Đảo màu).

Bản chất của Dò biên (Edge Detection)

-1	0	1
-2	0	2
-1	0	1

Sobel X (Dọc)

-1	-2	-1
0	0	0
1	2	1

Sobel Y (Ngang)



- Dò biên là tìm đạo hàm bậc nhất của ảnh.
- Sobel X: Phát hiện cạnh đứng.
- Sobel Y: Phát hiện cạnh ngang.



Bài tập 3: Thử thách Giải cứu ảnh (Denoise Challenge)

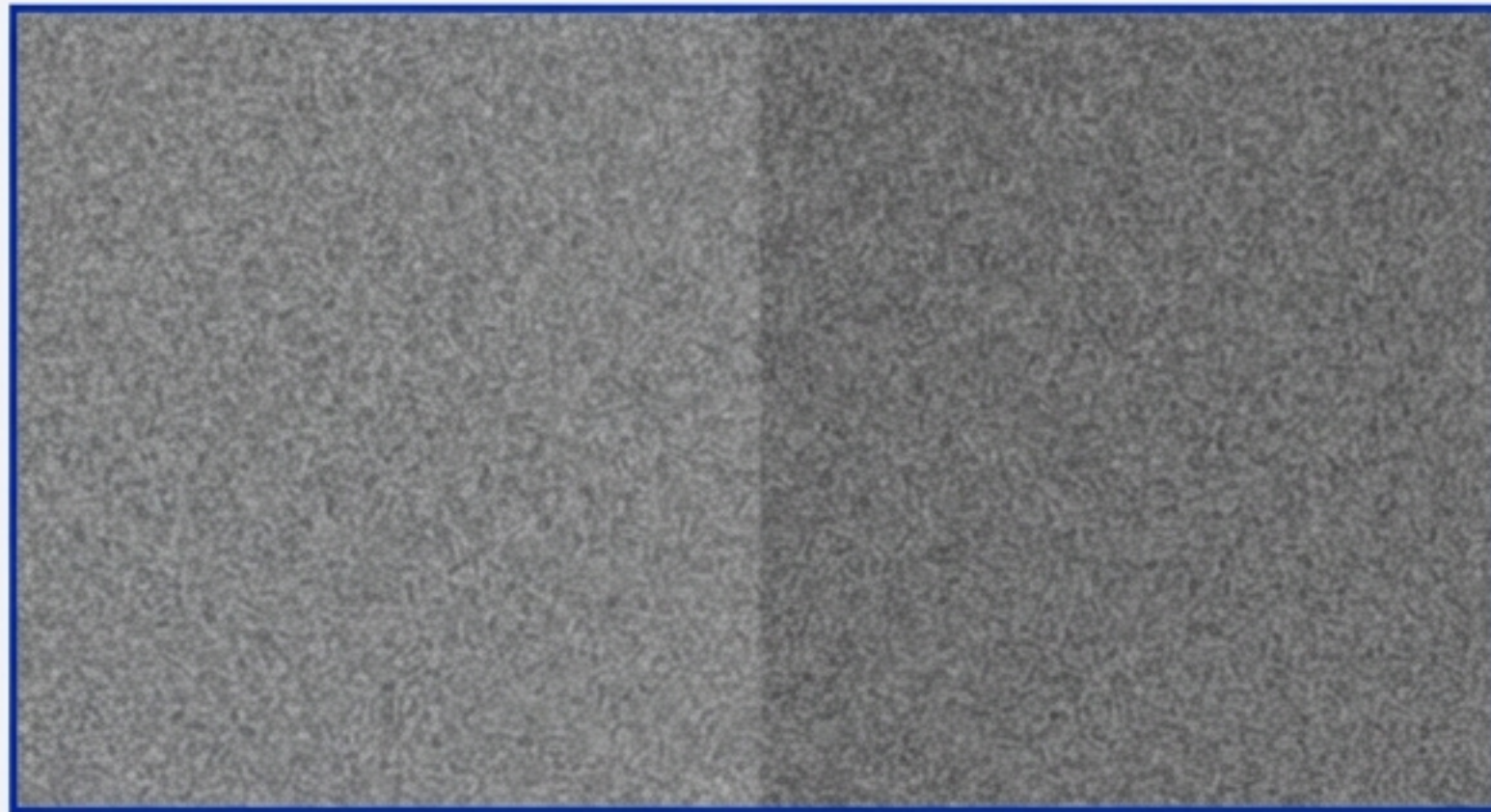
Vấn đề: Ảnh bị nhiễu hỗn hợp (Mixed Noise).

1. Nhiễu Muối tiêu.
2. Mờ nhòe (Blur).

Constraint: Không bộ lọc đơn lẻ nào xử lý được cả hai.

Phân tích Kẻ thù: Các loại nhiễu

Nhiều Gaussian



Sensor Noise. Biến động ngẫu nhiên.
Cần: Linear Filter (Gaussian Blur).

Nhiều Muối & Tiêu



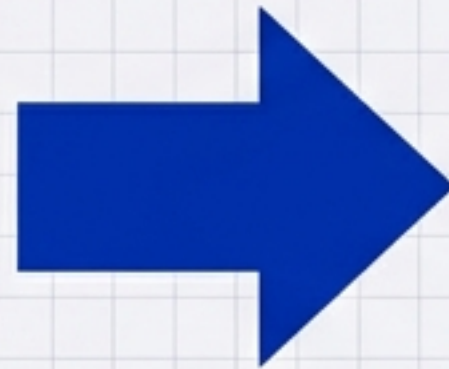
Impulse Noise. Điểm ảnh chết.
Cần: Non-linear Filter (Median Blur).

Cái bẫy chết người: Sai thứ tự

- Sai lầm: Dùng Gaussian Blur trước.
- Hậu quả: Hạt nhiễu bị hòa tan và lan rộng ra xung quanh.
- Nguyên tắc: Phải dọn rác (Median) trước khi lau sàn (Gaussian)!



Trước khi xử lý (Salt & Pepper)



Gaussian
Blur First



Sau khi Gaussian Blur (Sai)

Bước 1: Dọn rác (Median Blur)



- **Cơ chế:** Chọn giá trị trung vị (median) trong cửa sổ trượt.
- **Hiệu quả:** Loại bỏ hoàn toàn giá trị cực đoan (0/255) của nhiễu muối tiêu.

```
cv2.medianBlur(img, 3)
```

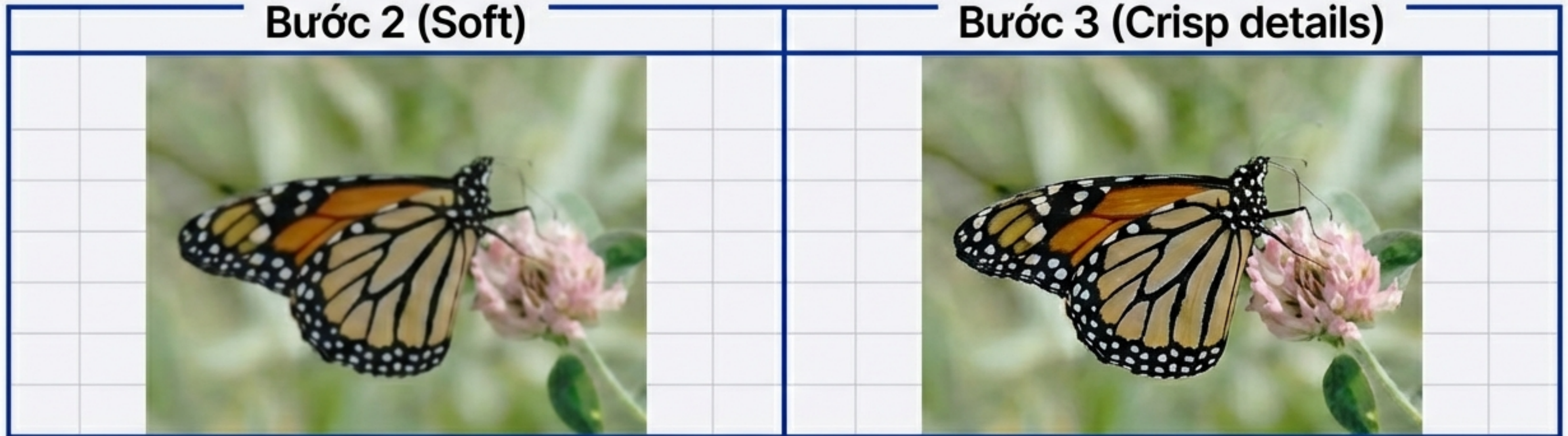

Bước 2: Làm mịn (Gaussian Blur)



- **Mục tiêu:** Khử nhiễu nền (background noise) và làm phẳng vùng màu.
- **Cơ chế:** Trung bình cộng có trọng số.

```
cv2.GaussianBlur(img_median, (5,5), 0)
```

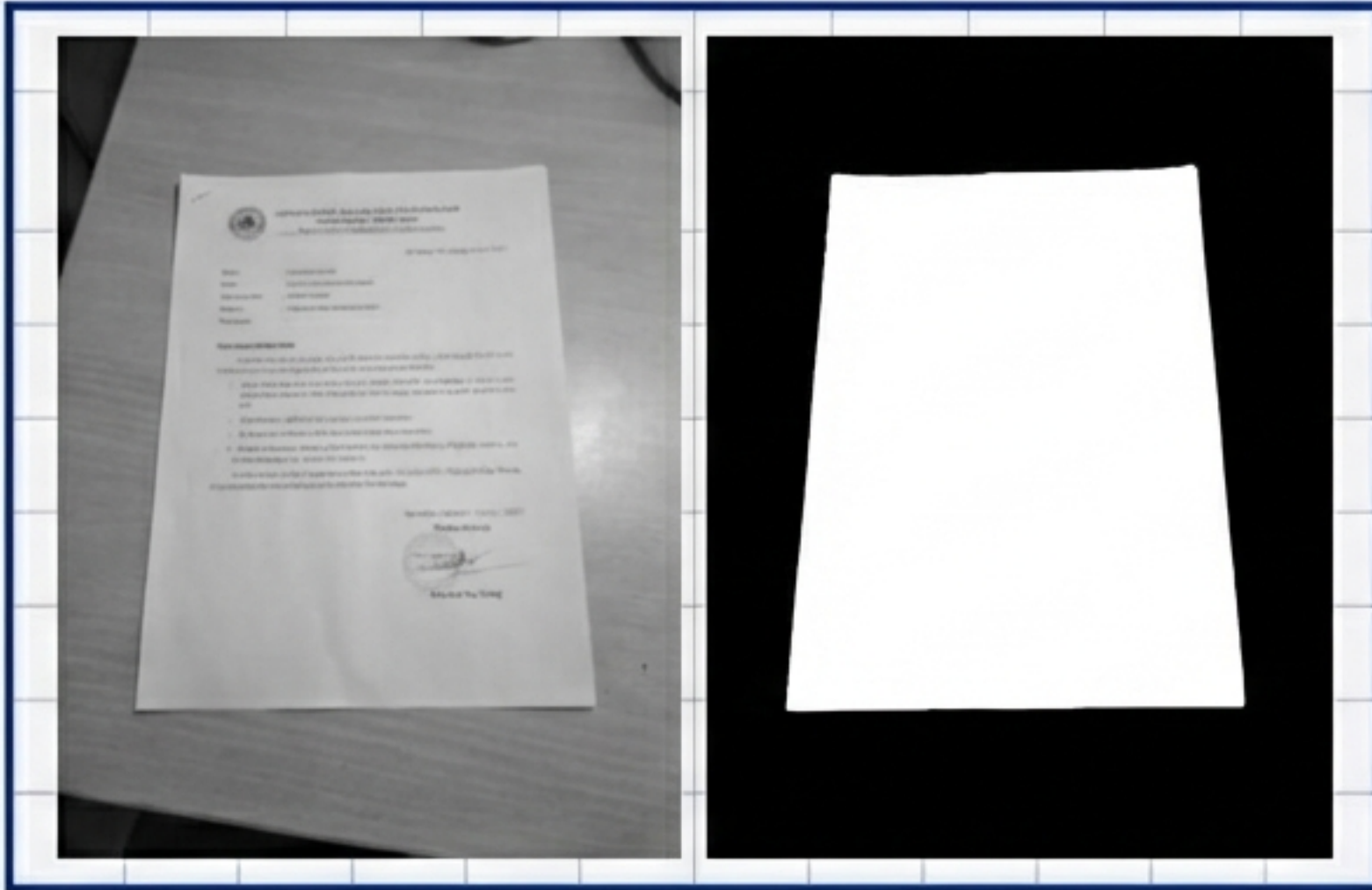

Bước 3: Lấy lại nét (Sharpening)



- **Tool:** `cv2.filter2D` + Custom Sharpen Kernel.
- **Pipeline Hoàn chỉnh:** Clean -> Smooth -> Recover.

```
cv2.filter2D(img_gaussian, -1, sharpen_kernel)
```

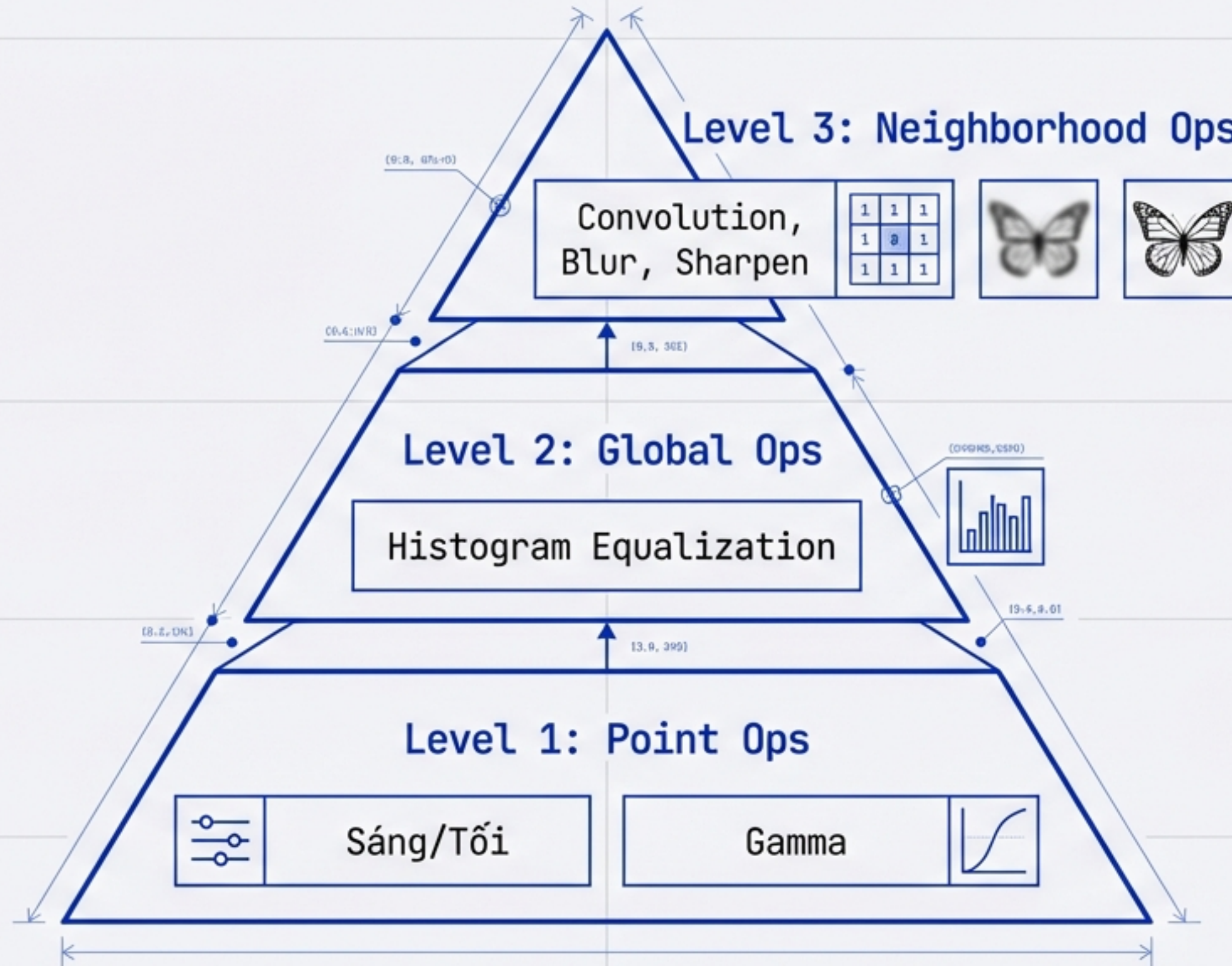

Mini-Project: Ứng dụng 'Document Scanner'



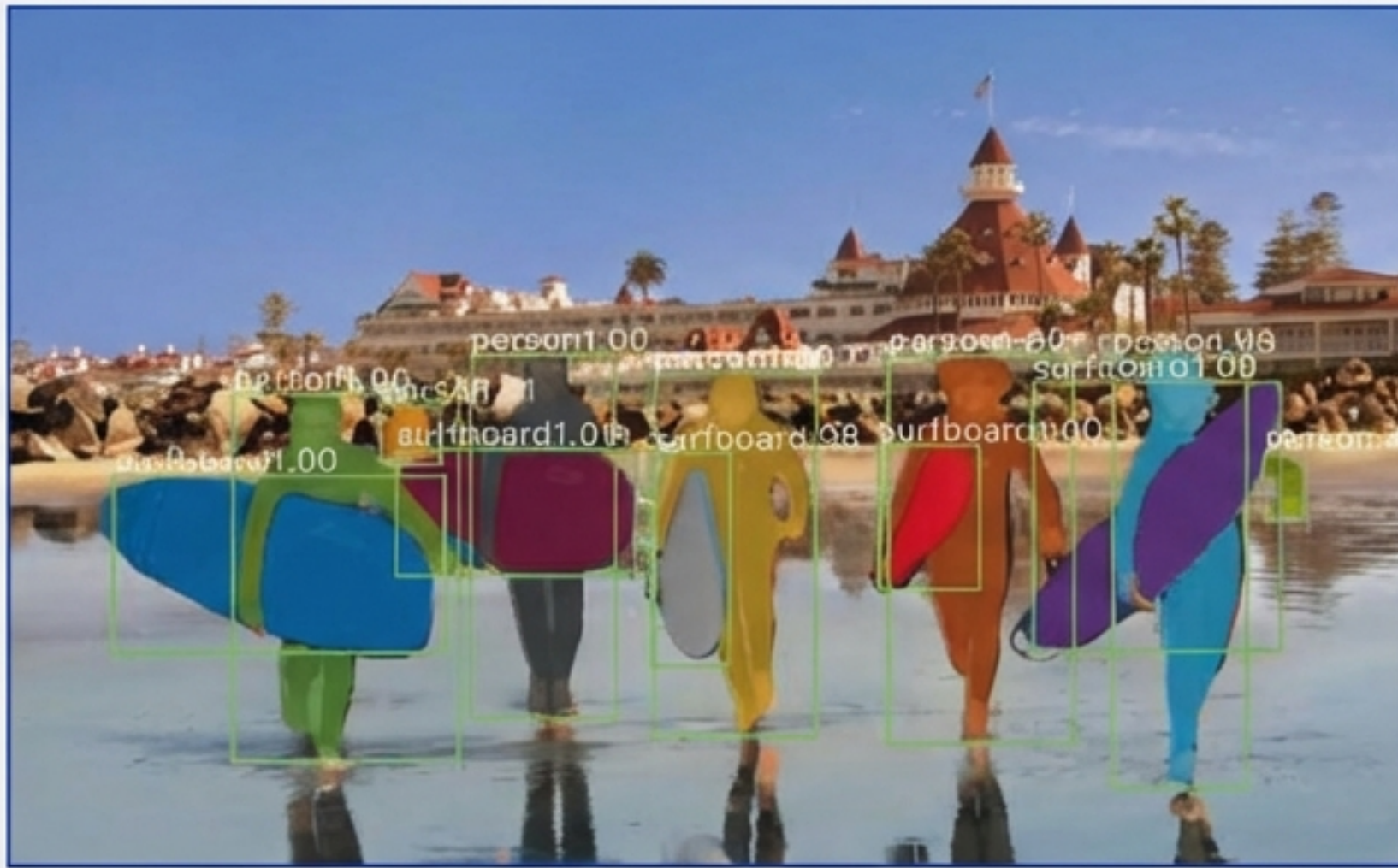
- **Mục tiêu:** Xây dựng tính năng scan tài liệu (CamScanner clone).
- **Pipeline yêu cầu:**
 1. Grayscale (Ảnh xám).
 2. Gaussian Blur (Lọc nhiễu mặt bàn).
 3. Edge Detection (Bắt 4 cạnh tờ giấy).

Nộp file Notebook trước buổi học tuần sau.

Tổng kết Chương 2: Bạn đã làm chủ được gì?



Hướng tới Chương 3: Phân đoạn ảnh (Segmentation)



Ví dụ: Semantic Segmentation trên ảnh người lướt sóng.
JetBrains Mono

Tổng quan

- **Trạng thái hiện tại:** Ảnh sạch, nét, tương phản tốt.
- **Vấn đề:** Máy tính chưa biết 'đâu là vật, đâu là nền'.
- **Nhiệm vụ tiếp theo:** Cắt ảnh thành các phần có ý nghĩa (Thresholding, K-Means).